Uttapreksha Patel

Boston, MA | patel.utt@northeastern.edu | (617)-961-9251 | GitHub | LinkedIn | Portfolio | Medium | Open to Relocate

ML infrastructure engineer who writes CUDA kernels from scratch and profiles them to the metal: built an LLM inference stack in CUDA C++ reaching 90% of cuBLAS and 0.91–0.95× of official FlashAttention-2, diagnosed every bottleneck with Nsight Compute and SASS inspection. Strong Python/C++ fundamentals, deploys and optimizes systems end to end.

Technical Skills

GPU/CUDA	CUDA C++, Tensor Cores (`mma.sync`, `ldmatrix`), Nsight Compute, SASS
Kernels	FlashAttention-2, Fused GEMM/RMSNorm/RoPE/SwiGLU, Quantized GEMV, MoE
ML/RL	PyTorch, GRPO/PPO/DPO (from scratch), Reward Design
Systems	Docker, Linux Process Isolation, Kubernetes, AWS EC2
Languages	Python (OOP, PEP8), C++ (pybind11), CUDA, SQL

Work Experience

Northeastern University | *Collaboration with Sandia National Laboratories* — **Boston, MA**
Research Assistant | Systems Engineering & Performance Diagnosis — *February 2025 – Present*

- Scoped and shipped Python data pipelines end-to-end without a fully specified brief, processing **1M+ operation records** with automated anomaly detection; deployed to production on AWS EC2, improving pipeline reliability **30%**
- Diagnosed performance problems at the resource level: designed controlled experiments isolating exactly which resource caused a performance regression, quantifying effects with sub-second precision across **8 identified events**
- Deployed telemetry infrastructure and diagnosed critical buffer-overflow failures through memory allocation tuning, restoring stable service under multi-node production load

TecUnique Pvt. Ltd. — **Vadodara, India**
Python Development Intern — *Dec 2023 – Apr 2024*

- Built Python REST API automation framework using Pytest achieving **93% pass rate across 30 test cases**; automated end-to-end validation with structured HTML reports and regression tracking
- Optimized SQL queries and numerical pipelines, reducing runtime **20%**; managed source control through Git in an Agile/Scrum environment

Relevant Projects

nanoserve: LLM Inference Kernels in CUDA C++, Built Entirely from Scratch | *CUDA C++, PyTorch, Nsight Compute, SASS | GitHub*

- Built an LLM inference stack from scratch in CUDA C++ (GEMM, FlashAttention-2, fused RMSNorm/RoPE/SwiGLU, 4-bit quantized GEMM, Mixture-of-Experts) and integrated it into PyTorch to run Qwen2.5-1.5B and 7B on an NVIDIA L4 GPU
- Achieved **2.61× faster** token generation than Hugging Face Transformers on Qwen2.5-7B (45.1 vs 17.3 tokens/s) with a W4A16 group-quantized GEMV streaming weights at **100% of DRAM bandwidth**, keeping BF16 perplexity within 0.02% of Hugging Face
- Implemented a FlashAttention-2 forward kernel (`ldmatrix`, `mma.sync`, `cp.async` pipelining, online softmax in registers, causal masking, GQA) reaching **67 TFLOP/s** and **0.91–0.95×** of official FlashAttention-2 at sequence lengths up to 2048; longest-first causal scheduling raised throughput up to **1.3×**
- Optimized a BF16 tensor-core GEMM to **90% of cuBLAS** and FP32 SGEMM from 1.7% to 91% across 14 kernel iterations, diagnosing each bottleneck with Nsight Compute and SASS inspection (e.g. replacing WMMA's generic shared-memory loads with `ldmatrix`: +19 percentage points)
- Built an MoE layer with device-side routing and a grouped GEMM; profiler-driven fixes (L2-aware tile rasterization, cache-line-sized tiles) gave a **1.8×** prefill speedup and **1.16×** over a per-expert cuBLAS loop for many-expert models

GPU Performance Benchmarking: Cross-Hardware Profiling, Nsight Analysis | *PyTorch, CUDA, NVIDIA Nsight, ResNet-50, ImageNet, MS COCO, CIFAR-100*

- Benchmarked ResNet-50 training across NVIDIA P100, V100, and H100; profiled CUDA kernel execution, SM occupancy, and memory bandwidth with NVIDIA Nsight to diagnose the actual bottleneck rather than trusting an aggregate number
- Identified **up to 70% lower training time** on H100 vs. P100, isolating Tensor Core utilization and NVLink bandwidth as the real drivers; built repeatable automated profiling pipelines for systematic performance analysis

GRPO-LLM: RL Post-Training, Custom C++ Extensions, Kubernetes Deployment | *Python, PyTorch, C++/pybind11, Kubernetes, AWS | GitHub | Blog*

- Implemented GRPO from scratch without wrapping TRL; ran a controlled 3-way comparison against PPO and DPO, isolating reward sparsity, not compute, as the actual bottleneck
- Profiled the training loop, found the real hot path, and wrote a custom C++ extension measuring a **1.7× speedup**, validated with **25 passing tests**, deployed via Kubernetes

3D Point Cloud Object Detection (KITTI): Hardware-Level Debugging | *Python, PyTorch, NumPy, AWS EC2, CUDA | GitHub*

- Found and fixed a GPU-CPU sync bottleneck caused by **240,000 per-scene `.item()` calls**, rewriting the hot path fully vectorized; achieved a **15.6×** near-field AP improvement across 140 epochs

Research

Analyzing VLM-Based Approaches for Anomaly Classification and Segmentation | *Co-authored, arXiv 2026*

- Evaluated WinCLIP and AnomalyCLIP on MVTec AD and VisA benchmarks; achieved **0.916 classification AUROC** and **0.907 segmentation AUROC** through reproducible evaluation pipelines and systematic error analysis

Education

Northeastern University — **Boston, MA**
M.S. Electrical and Computer Engineering (GPA: 3.9/4.0) — *May 2026*

G H Patel College of Engineering and Technology — **Anand, India**
B.E. Electronics and Communication Engineering (GPA: 3.7/4.0) — *May 2024*